

DIPT

SYSTEM HANDBOOK

Overview

How the system is shaped

The six-stage pipeline

The status lifecycle

The agents

The database

Shared infrastructure

The three surfaces

Local models

What was built

Starting point

Summarisation agent

Scoring agent

QA agent

Master pipeline & streaming

Dashboard

Scheduler

Public website

Added beyond the brief

Hardware problems & fixes

Tests

Running the system

Prerequisites & setup

Starting the pieces

The dashboard, view by view

The command line

The automated schedule

The QA demo

Building the website

First-run walkthrough

[Configuration reference](#)

[Current state & demo data](#)

[Known limitations & follow-ups](#)

[File-by-file reference](#)

[Troubleshooting](#)

[Glossary](#)

DIPT System Handbook

Everything the pipeline is, everything that was built, and exactly how to run it — written for a reader who has not seen the code or the process before.

7 pipeline stages

6 agents

162 automated tests

28 approved papers loaded

runs fully offline

1 Overview

DIPT is an automated monitoring line for Software Engineering (SE) research papers, built for Professor Tony Gorschek at SERL Sweden, Blekinge Institute of Technology. It runs entirely on one local workstation (an HP Z2 Mini with an AMD Ryzen AI Max Pro 395 and 96 GB of unified memory) using local language models served by Ollama. There is no cloud service and no per-use cost.

The system is an **assembly line**. A paper enters as nothing more than a title and an abstract. It comes out the other end fully read, sorted into subject categories, summarised in a fixed four-part structure, and given an industrial-relevance score with a written justification. Each step is handled by a separate program called an *agent*. Every paper carries a `status` field in the database that records how far along the line it has travelled, so any agent can stop and later resume without losing work.



Three of the six agents (Fetch, Parse, Categorise) already existed when this work began. The remaining components — Summarise, Score, QA, the orchestration around them, an operator dashboard, an automated scheduler, and a public website — were built during this engagement, along with several conveniences that were not in the original brief. Section 3 documents all of it.

Where things stand now: the code is complete and covered by 162 automated tests. The database holds 28 papers that have been carried all the way to **approved**, and the public-site data files have been generated from them. The system is ready to demonstrate and ready to run on a schedule.

2 How the system is shaped

This section is the mental model. Read it once and the rest of the handbook will make sense.

2.1 The six-stage pipeline

A paper moves left to right. Each stage reads papers that are in the status it expects, does its work, writes its output to the database, and advances the paper's status. Stages never skip ahead; a paper only reaches *Score* once *Summarise* has finished with it.

#	STAGE	WHAT IT DOES	READS PAPERS AT	LEAVES THEM AT
1	Fetch	Pulls recent open-access SE papers from seven sources (OpenAlex, arXiv, Semantic Scholar, DBLP, CrossRef, Zenodo, CORE). Every candidate must pass an LLM "is this really SE research?" gate before it is saved.	—	fetches
2	Parse	Downloads each paper's PDF, checks it is a real PDF, extracts the full text with PyMuPDF, and writes the text to disk.	fetches	parsed / no_pdf / no_text
3	Categorise	Assigns each paper to 1–3 of 47 SE categories, each with a confidence score. The category list is read live from the database, so it can be edited without touching code.	parsed / no_pdf	categorised
4	Summarise	Produces a structured summary with four labelled parts: research problem, methodology, key findings, industrial implications. Uses the full text when available, the abstract otherwise.	categorised	summarised
5	Score	A reasoning model acts as a "virtual CTO" and gives the paper an industrial-relevance score from 1 to 10 plus a written rationale aimed at a technical executive.	summarised	scored
6	QA	Re-checks the summary and score for consistency and obvious errors. If a problem is found it first tries to fix it automatically by regenerating the affected output; only if that fails does it raise a flag for a human. Papers that pass reach approved.	scored	approved (or a QA flag)

2.2 The status lifecycle

Every paper row has exactly one `status`. These are all the possible values and how a paper moves between them.

```

fetched → parsed → categorised → summarised → scored → approved
|
|   no_pdf → categorised      (rejected is available but unused today)
|   no_text (dead end: no usable text and no abstract)
└─ (a paper is never deleted; it only ever moves forward)
```

STATUS	MEANING
<code>fetched</code>	Saved by Fetch, passed the SE gate, nothing else done yet.
<code>parsed</code>	PDF downloaded and full text extracted to disk.
<code>no_pdf</code>	No usable PDF was found. The paper still continues — later stages fall back to the abstract.
<code>no_text</code>	A PDF was found but yielded too little text. Dead end unless a human intervenes.
<code>categorised</code>	Has 1–3 category assignments.
<code>summarised</code>	Has a four-part summary stored.
<code>scored</code>	Has an industrial-relevance score and rationale stored.
<code>approved</code>	Passed QA. This is the only status that appears on the public website.
<code>rejected</code>	Defined in the schema for completeness; not produced by any current stage.

2.3 The agents

An "agent" is just a Python class with a `run()` method. Each one is small, single-purpose, and independently testable. They share no global state — everything an agent needs is handed to it when it is created.

- **Fetch agent** — loops over the configured sources, passes every candidate paper through the quality gate, and saves the survivors. Duplicates (same source identifier) are skipped silently.

- **Parser agent** — for each fetched paper, downloads the PDF over HTTP with an explicit timeout, verifies the file really is a PDF, extracts and cleans the text, and records where it was written.
- **Categorisation agent** — loads the 47 active categories from the database, asks the model to pick the best-fitting ones with confidences, validates every returned name against the known list (with a fuzzy fallback), and keeps the top three.
- **Summarisation agent** — builds a prompt from the paper's title plus up to ~9,000 characters of full text (or the abstract), asks the model for a JSON object with four keys, and stores the rendered result. See 3.2.
- **Scoring agent** — feeds the stored summary to a reasoning model with a scoring rubric, strips the model's private "thinking" section, parses out the score and rationale, and stores them. See 3.3.
- **QA agent** — the only agent with two modes. It reviews the stored summary and score; on a problem it regenerates the weaker of the two once and re-reviews; if it still fails, it writes a row to the `qa_flags` table for a person to look at. See 3.4.

A seventh helper, the **quality gate**, is not a pipeline stage but is used by Fetch: it is a fast yes/no classifier that fails closed (when in doubt, or on error, it answers "no").

2.4 The database

PostgreSQL, database name `dipt`, accessed by the application role `dipt_user`. Six tables were part of the original design; a seventh (`fetch_log`) is optional and only the scheduler uses it.

TABLE	HOLDS
<code>papers</code>	One row per paper: title, abstract, authors, DOI, source, a unique source identifier, PDF URL, publication date, <code>status</code> , and the path to the extracted text file.
<code>categories</code>	47 rows — 18 SWEBOK v4 knowledge areas plus 29 emerging SE domains — each with a name, description, and an <code>is_active</code> flag. Editable from the dashboard.
<code>paper_categories</code>	Which categories a paper was assigned, with a confidence per assignment.
<code>summaries</code>	One summary per paper: the rendered four-part text, the model used, and a timestamp.
<code>scores</code>	One score per paper: the 1–10 value, the rationale, the model used, and a timestamp.
<code>qa_flags</code>	One row per unresolved QA problem: the paper, a short flag type, a description, and a <code>resolved</code> flag.
<code>fetch_log</code> <code>optional</code>	One row per scheduled run: start time, finish time, papers saved, and success/failure. Used only to narrow the next fetch window and to show run history. Created by the migration in <code>migrations/</code> .

All SQL lives in one file (`dipt/database/repository.py`). Agents never write SQL; they call methods on a repository object. Every query is parameterised, so there is no SQL-injection surface.

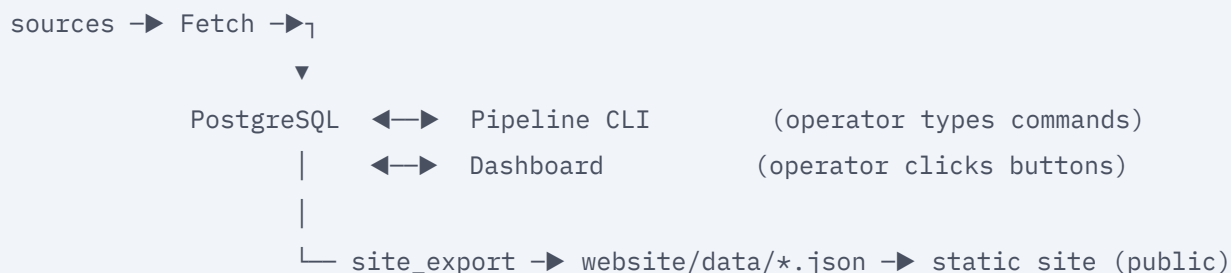
2.5 Shared infrastructure

- **Configuration** (`dipt/config.py`) — all settings come from a `.env` file, validated at start-up. A missing database password, an out-of-range value, or a bad log level stops the program immediately with a clear message.
- **Exceptions** (`dipt/exceptions.py`) — every custom error inherits from `DIPTError`, so the entry points can catch the whole family in one place while still handling specific failures where it matters.
- **Logging** (`dipt/logging_config.py`) — one central set-up; every module logs, nothing prints.

- **Connection pool** (`dipt/database/connection.py`) — a small pool of database connections, handed out through a context manager that commits on success and rolls back on any error.
- **LLM client** (`dipt/agents/llm_client.py`) — the single wrapper around Ollama. It adds retries with exponential back-off, an optional token cap, and an optional "JSON mode" that forces the model to return valid JSON. Every agent talks to models only through this.
- **Prompt store** (`dipt/prompt_store.py`) — lets an administrator override any agent's prompt by dropping a text file in `config/agent_prompts/`. Added during this work; see 3.9.
- **Typed models** (`dipt/models/schemas.py`) — Pydantic models form the boundary between raw external data and internal logic: `PaperRecord`, `Category`, `CategoryAssignment`, `PaperSummary`, `PaperScore`, `QAReview`, plus the `PaperStatus` and `PaperSource` enumerations.

2.6 The three surfaces

All three read and write the same PostgreSQL database. They are independent — you can use any subset.



- **Pipeline CLI** — `python -m dipt.pipeline <stage> [N]`. The original way to drive the line. Still fully supported.
- **Dashboard** — a Streamlit web app for the SERL team. Runs stages as background jobs, shows pipeline status, browses papers, works the QA queue, edits categories and prompts, and turns the automated schedule on and off. Built during this work.
- **Public website** — a static Next.js + Fuse.js site of approved papers, for the outside world. Data is exported from the database by a small script. Built during this work.

2.7 Local models

Served by Ollama on `http://localhost:11434` . All chosen so the whole line runs on the one workstation with no external calls.

STAGE	MODEL	WHY
Quality gate	<code>llama3.1:8b</code>	Small and fast; only needs to answer YES/NO.
Categorisation	<code>llama3.1:70b</code>	Enough capability to map a paper onto 47 fine-grained categories.
Summarisation	<code>qwen2.5-coder:32b</code>	Substituted for the brief's <code>command-r-plus</code> , which will not load on this hardware. See 3.10.
Scoring	<code>deepseek-r1:70b</code>	A reasoning model — it "thinks" before answering, which suits the virtual-CTO judgement task.
QA review	<code>llama3.1:70b</code>	Capable general model; the review is a short structured check.

3 What was built

This section is the complete work log: what existed before, what was delivered, the design decisions inside each piece, and the problems that came up along the way.

3.1 Starting point

When this engagement began, the repository already contained a production-standard rebuild of the first three agents — Fetch, Parse, Categorise — plus all the shared infrastructure listed in 2.5, and **39 passing tests**. The database already had the 47 categories and a backlog of roughly 700 papers at `parsed` or `no_pdf` . Nothing in that baseline was rewritten; the new work was added on top and matched to the existing style (type hints everywhere, Google-style docstrings, custom exceptions, no bare `except` , explicit network timeouts, all SQL in the repository).

Six items remained: the Summarisation agent, the Scoring agent, the QA agent, the pipeline orchestration and a streaming mode, the dashboard, the scheduler, and the public website.

3.2 Summarisation agent

A partial implementation existed (`dipt/agents/summarisation_agent.py`). It was finished and corrected:

- **Full-text bug fixed.** The agent only read a paper's extracted text when the paper's status was exactly `parsed` . But by the time a paper reaches this stage its status is `categorised` , so that check was never true and every summary was built from the abstract alone. It now uses the full text whenever a text file exists, regardless of status.
- **Switched to JSON mode.** The original prompt asked the model for nested XML tags. Local models on this box comply with that poorly — and on longer inputs `llama3.1:70b` would collapse into repeated phrases. The agent now asks for a JSON object with four string keys and turns on Ollama's JSON mode, which constrains the output to valid JSON. Parsing is lenient: unknown keys are ignored, missing keys become empty, a stray code fence is stripped.
- **Model changed** from `command-r-plus` to `qwen2.5-coder:32b` — see 3.10 for the full reasoning.
- **Input capped** at ~9,000 characters and output at ~800 tokens, both to keep the model stable and to keep each paper to under a minute or two.

It was wired into the pipeline (`summarise` command) and covered by 14 new tests. Verified live: 28 papers summarised with no failures.

3.3 Scoring agent

New file `dipt/agents/scoring_agent.py` . It selects `summarised` papers, sends the stored summary to `deepseek-r1:70b` with a scoring rubric, removes the model's `<think>...</think>` block, and extracts a 1–10 score plus a three-to-five-sentence rationale written for a technical executive. Out-of-range scores are clamped rather than rejected. Saved to the `scores` table; status advances to `scored` . Wired as the `score` command, 10 new tests, verified live on 28 papers.

The rubric was later tuned (sharper bands, an explicit instruction to use the whole range and justify why the number is not one point higher or lower, sampling temperature raised from 0.1 to 0.35) to counter a tendency of the model to give almost every paper the same number. See 6.

3.4 QA agent

New file `dipt/agents/qa_agent.py`. Professor Gorschek's specific requirement was that QA must have **two modes**, and that the auto-fix mode — the one that usually gets skipped — must actually work. Both are implemented and both were demonstrated on real papers.

1. **Review.** The agent loads the stored summary and score and asks `11ama3.1:70b` (JSON mode) to fail the paper if the summary is empty, truncated, or off-topic, or if the score is missing, out of range, or plainly inconsistent with its own rationale. The verdict names which output is weaker: `summary` or `score`.
2. **Auto-fix, then escalate.** On a pass, the paper goes straight to `approved`. On a fail, the agent regenerates the named output *once* (calling the Summarisation or Scoring agent), then re-reviews. If it now passes, the paper is approved and the run is recorded as an auto-fix. If it still fails — or if regeneration produced nothing — the agent writes a row to `qa_flags`, leaves the paper at `scored`, and moves on. A flagged paper is excluded from future QA runs until a person clears the flag from the dashboard.

Wired as the `qa` command, 14 new tests covering every branch. A helper script, `scripts/qa_demo.py`, reproduces both paths on demand for a demonstration:

COMMAND	WHAT IT SHOWS
<code>python -m scripts.qa_demo --autofix <id></code>	Corrupts a paper's stored summary, then shows QA detect it, regenerate the summary, re-check, and approve.
<code>python -m scripts.qa_demo --escalate <id></code>	Corrupts the summary and forces regeneration to fail, then shows QA give up and write a <code>qa_flags</code> row visible in the dashboard QA queue.

Live results on the demo batch: 28 of 28 passed review, and the auto-fix path fired once organically on a real paper.

3.5 Master pipeline & streaming

`dipt/pipeline.py` was extended:

- `score`, `qa`, and `stream` were added to the command list.

- `all` now runs the whole chain in order (`fetch → parse → categorise → summarise → score → qa`) in one command.
- **Streaming mode** (`stream`) takes papers that are `categorised` and carries each one individually through `summarise → score → QA` before moving to the next paper. In batch mode you would wait for every paper to be summarised, then every paper scored, then every paper QA'd, before any single paper is approved; streaming makes approved papers appear from the first paper onward. Fetch/parse/categorise stay batch (they are network-bound or cheap).

7 tests for the wiring and the streaming loop.

3.6 Dashboard

A Streamlit app under `dashboard/`, run with `streamlit run dashboard/app.py`. Six views (full operating instructions in 4.3):

VIEW	PURPOSE
Run pipeline	Start any stage as a background job with a paper cap; watch its live log; stop it. One run at a time.
Schedule	Turn the automated full-pipeline run on/off; set the interval (default 24 h) and an optional per-cycle cap; run one cycle now; see the next-run time.
Pipeline status	Paper counts per lifecycle stage, approved / open-flag totals, and recent scheduled runs.
Paper browser	Filter papers by status; open one to see its categories, four-part summary, score, and rationale.
QA queue	Every escalated paper with its stored summary and score, and a "Mark resolved" button.
Settings	Add / rename / describe / deactivate the 47 categories, and edit any agent's prompt.

Two implementation notes. The dashboard deliberately renders Markdown tables and text bars instead of `st.dataframe` / `st.bar_chart`, because the charting library's binary component is blocked by an application-control policy on the production machine. And all database access still goes through the one repository class — the dashboard adds no SQL of its own.

3.7 Scheduler

`dipt/scheduler.py` plus `docs/SCHEDULING.md` and a one-line SQL migration. A scheduled run does the full chain, but first reads the last successful run's start time and narrows the fetch window to "since then plus one day of overlap" so it does not re-examine papers it has already seen. It records the run in `fetch_log` and refreshes the website data at the end.

`docs/SCHEDULING.md` gives copy-paste setup for Windows Task Scheduler (the production route) and for cron / launchd. 8 tests.

3.8 Public website

A complete static Next.js (pages router, static export) + Fuse.js project under `website/`, plus `dipt/site_export.py` which reads every `approved` paper and writes three files the site consumes at build time:

- `website/data/papers.json` — the full records, including the parsed four-part summary, score, and categories.
- `website/data/categories.json` — the categories that have at least one approved paper, with counts.
- `website/public/rss.xml` — an RSS 2.0 feed of the most recent approved papers.

Pages: a home page with client-side search (Fuse.js) plus category and minimum-score filters; a detail page per paper; a categories index; a page per category; and the RSS feed. What a visitor sees is described in 4.7. 8 tests for the exporter.

NOT BUILT HERE

The website's production build (`npm run build`) has not been run — this machine has no Node.js. The project and the exported data are complete; the build must be done on a machine with Node before deploying.

3.9 Added beyond the brief

- **Fetch cap.** `fetch` now takes a number: `python -m dipt.pipeline fetch 20` stops after saving 20 new papers. Flows through `all` and the dashboard too.

- **Dashboard "Run pipeline" view.** Runs each stage as a detached background process, tracked by files under `runs/`. You can close the browser; the run keeps going. A "recent runs" table shows status and duration; a log pane tails the output; a Stop button kills it.
- **Dashboard "Schedule" view.** A small daemon process runs the whole pipeline every N hours (default 24), started and stopped from the UI. Survives a browser refresh or a Streamlit restart; does not survive a machine reboot (use the OS scheduler for that). Optional per-cycle paper cap. Respects the "one run at a time" rule.
- **File-based prompt overrides.** An administrator can rewrite any agent's prompt from the Settings view. Overrides are saved as plain text files under `config/agent_prompts/` and picked up on the next run — no database table, no migration, no special privileges. A save is blocked if it drops a required `{placeholder}`.

3.10 Hardware problems & fixes

The workstation's integrated GPU (Radeon 8060S, ~76 GiB usable) is powerful but has real limits with the largest models. Four issues came up; all are resolved and the fixes are persisted.

PROBLEM	CAUSE	FIX
<code>command-r-plus</code> (the brief's summarisation model) never starts	59 GB of weights load, then the model server times out allocating the key/value cache.	Summarisation now uses <code>qwen2.5-coder:32b</code> , which produces clean structured summaries and leaves memory headroom. One-line revert in <code>.env</code> when the GPU memory ceiling is raised.
<code>llama3.1:70b</code> produced repeated-phrase gibberish for summaries	The 70B model degrades on a long prompt combined with a fragile nested-XML output format.	All structured-output agents switched to JSON mode with lenient JSON parsing, and inputs were capped.
Scoring failed for 23 of 28 papers mid-run	GPU memory exhaustion after several back-to-back 70B model loads; the model server could not allocate a 30 GB cache and started returning empty output.	Ollama restarted with a smaller context window and a quantised key/value cache (roughly one-eighth the memory). The re-run then completed 23/23 with no failures.
Model loads timed out under memory pressure	Ollama's default 5-minute load timeout is too short for a 42 GB model when memory is tight.	Load timeout raised to 20 minutes; keep-alive raised to 30 minutes so a model stays resident for a whole stage.

The five Ollama settings that fix this are now saved to the Windows user environment, so the Ollama app and any scheduled run pick them up automatically:

```
OLLAMA_LOAD_TIMEOUT    = 20m
OLLAMA_KEEP_ALIVE      = 30m
OLLAMA_CONTEXT_LENGTH  = 8192
OLLAMA_KV_CACHE_TYPE   = q8_0
OLLAMA_FLASH_ATTENTION = 1
```

NOTE

During this work the Ollama tray app was stopped and a server was run inside the work session. Before relying on the system, launch **Ollama** from the Windows Start menu once — it will start with the settings above.

3.11 Tests

The suite grew from **39** to **162**. Every language-model and database call is mocked, so the tests run in about one second and need neither Ollama nor PostgreSQL. Run them with:

```
python -m pytest tests/ -v
```

TEST FILE	COVERS	TESTS
test_summarisation_agent.py	JSON parsing, full-text vs abstract, status advance	14
test_scoring_agent.py	score extraction, think-block stripping, clamping	10
test_qa_agent.py	pass / auto-fix / escalate branches	14
test_pipeline.py	command dispatch, streaming loop, prompt lookup	7
test_scheduler.py	fetch-window maths, run bookkeeping	8
test_site_export.py	slugs, summary parsing, RSS, file output	8
test_prompt_store.py	file overrides, validation, reset	9
test_runner.py	background-run status from log files	8
test_schedule.py	daemon config, enable / disable, status	11
test_fetch_agent.py	the new save cap	4
test_llm_client.py	retry, JSON mode, token cap	6
test_repository.py + test_repository_support.py	every SQL method, result mapping	24
test_categorisation_agent.py , test_quality_gate.py , test_sources.py , test_config.py , test_parser_agent.py	the original baseline, still green	33

4 Running the system

Written for someone starting from nothing. Every command is run from the project folder:

```
cd C:\Users\sepro\Downloads\dipt_production\dipt_build
```

4.1 Prerequisites & one-time setup

1. **Python packages.** Once:

```
python -m pip install -r requirements.txt
```

2. The **.env** file already exists in the project folder with the database password and paths filled in. If you ever recreate it, copy **.env.example** and edit it. Full reference in 5.

3. **PostgreSQL** must be running with a database called **dipt**. Check:

```
psql -U postgres -d dipt -c "SELECT 1"
```

4. **Ollama** must be running. Launch **Ollama** from the Start menu, then check:

```
curl http://localhost:11434/api/tags
```

5. **Optional migration.** Only if you want the scheduler's run history and incremental fetch, run this once as the database owner:

```
psql -U postgres -d dipt -f migrations/001_support_tables.sql
```

Everything else — the pipeline, the dashboard, editing prompts, the website — works without it.

4.2 Starting the pieces

You need PostgreSQL and Ollama up (above). Then start the dashboard:

```
python -m streamlit run dashboard/app.py
```

It prints a URL and opens your browser at `http://localhost:8501`. Stop it with `Ctrl + C` in the terminal. If port 8501 is busy, add `--server.port 8600`.

You do not need the dashboard to run the pipeline — the command line (4.4) does everything too.

4.3 The dashboard, view by view

Pick a view from the left sidebar.

Run pipeline

1. Choose a **Stage**: `fetch`, `parse`, `categorise`, `summarise`, `score`, `qa`, `all`, or `stream`.
2. Set the **cap** (the number field). For `fetch` it limits how many new papers are saved; for everything else it limits how many papers the stage processes. Start small — the 70B models take 2–3 minutes per paper.
3. Click **Start run**. It launches in the background; you can leave the page.
4. Watch **Run log** lower down; click **Refresh** to update it. A run shows as  running,  finished,  failed, or  stopped. **Stop this run** kills a running job.

Only one run at a time — a second Start is refused while one is going.

Schedule

1. Set **Interval (hours)** — default **24**.
2. Optionally set **Papers per cycle** (0 = the whole backlog).
3. Click **Enable / restart**. A background daemon starts and runs the first full cycle immediately, then repeats every N hours.
4. The status row shows ON/OFF, the interval, the cap, whether the daemon is alive, the last cycle's outcome, and the estimated next run.
5. **Disable** stops the daemon and any cycle in progress. **Run one cycle now** fires a single cycle without changing the schedule.

REBOOT

The daemon does not restart after the machine reboots. For a setup that survives reboots, use Windows Task Scheduler as described in `docs/SCHEDULING.md`. Use one or the other, not both.

Pipeline status

Read-only. Three headline numbers (papers total, approved, open QA flags), a bar chart of how many papers sit at each status, and — if the migration was run — a table of recent scheduled runs.

Paper browser

Filter by status and row count, get a table, then pick a paper to see everything about it: authors, DOI, status, its 1–3 categories with confidences, the industrial-relevance score and rationale, the four-part summary, and the original abstract.

QA queue

One card per paper the QA agent escalated. Each shows the flag type, description, and the stored summary and score. **Mark resolved** clears the flag so the next QA run re-checks that paper. Empty means every scored paper passed.

Settings

Categories: a table of all 47, an "Add a category" form, and an "Edit or deactivate" panel — changes take effect on the next categorisation run with no restart. **Agent prompts:** pick an agent (quality gate, categorisation, summarisation, scoring, QA), edit its prompt, and **Save prompt** (blocked if you drop a required `{placeholder}`) or **Reset to default**. Saved prompts are files under `config/agent_prompts/` and apply on the next run.

4.4 The command line

Everything the pipeline does, without the dashboard.

```
python -m dipt.pipeline fetch 20          # fetch, save at most 20 new papers
python -m dipt.pipeline parse 20         # download + extract text for up to 20
python -m dipt.pipeline categorise 20    # assign categories to up to 20
python -m dipt.pipeline summarise 20     # summarise up to 20
python -m dipt.pipeline score 20        # score up to 20
python -m dipt.pipeline qa 20           # QA-check up to 20
python -m dipt.pipeline all 20          # every stage in order, cap 20 each
python -m dipt.pipeline stream 20       # up to 20 papers, each taken fully
                                         #   through summarise -> score -> qa
```

The number is optional; leaving it off processes the whole backlog. After a run that produced new approved papers, refresh the website data:

```
python -m dipt.site_export
```

4.5 The automated schedule

Two independent options; pick one.

- **Dashboard toggle** — the Schedule view (4.3). Easiest; good while the workstation stays on. Does not survive a reboot.
- **Windows Task Scheduler** — reboot-proof, headless, recommended for production. The exact `schtasks` command and a wrapper batch file are in `docs/SCHEDULING.md`. It runs the same entry point:

```
python -m dipt.scheduler
```

4.6 The QA demo

To show Professor Gorschek both QA behaviours on demand (pick any approved or scored paper id, e.g. from the Paper browser):

```
python -m scripts.qa_demo --autofix 942   # detect -> regenerate -> approve
python -m scripts.qa_demo --escalate 943  # detect -> can't fix -> raise a flag
```

After `--escalate`, open the dashboard's QA queue to see the flag; "Mark resolved" clears it.

4.7 Building the website

Needs Node.js, which is not on this machine. On any machine with Node:

```
cd website
npm ci
npm run dev      # preview at http://localhost:3000
npm run build    # produces website/out/ -- a fully static site
```

Deploy `website/out/` to any static host. For Cloudflare Pages: connect the repository, build command `npm run build`, output directory `website/out`, root directory `website`. A commit that changes `website/data/papers.json` triggers a rebuild, so the site refreshes whenever the pipeline approves new papers.

What a visitor sees:

PAGE	CONTENT
Home (<code>/</code>)	A count of approved papers, a live search box (matches titles, topics, findings, authors), a category filter, a minimum-score slider, and a card per paper — title, authors, date, source, a colour-coded score pill, the research-problem sentence, and clickable category tags.
Paper (<code>/papers/<id>/</code>)	Full title and metadata, DOI link, the score out of 10 with its rationale, category tags, the four summary sections, and the original abstract in a fold-out.
Categories (<code>/categories/</code>)	Every category that has at least one approved paper, with its count.
Category (<code>/categories/<slug>/</code>)	All approved papers in that category, highest score first.
RSS (<code>/rss.xml</code>)	The 50 most recent approved papers — title, link, the "why a practitioner should care" line, publication date.

Only `approved` papers ever reach the public site.

4.8 First-run walkthrough

If you are starting with an empty or near-empty database and want to see the whole line work end to end, on a small scale:

```
python -m dipt.pipeline fetch 5
python -m dipt.pipeline parse 5
python -m dipt.pipeline categorise 5
python -m dipt.pipeline summarise 5
python -m dipt.pipeline score 5
python -m dipt.pipeline qa 5
python -m dipt.site_export
```

Expect roughly 20–40 minutes, almost all of it in `score`. Then open the dashboard's Paper browser (filter: `approved`) to see the results, and the Pipeline status view to see the counts move. Or do the same thing from the dashboard's Run pipeline view, one stage at a time.

5 Configuration reference

5.1 The `.env` file

Sits in the project folder. Loaded and validated at start-up. Never commit it — it holds the database password.

SETTING	DEFAULT / CURRENT	MEANING
DB_HOST	localhost	PostgreSQL host.
DB_PORT	5432	PostgreSQL port.
DB_NAME	dipt	Database name.
DB_USER	dipt_user	Application role.
DB_PASSWORD	—	Set in the file. Required.
DB_POOL_MIN / DB_POOL_MAX	1 / 5	Connection-pool bounds.
OLLAMA_BASE_URL	http://localhost:11434	Where Ollama listens.
MODEL_QUALITY_GATE	llama3.1:8b	SE-relevance gate.
MODEL_CATEGORISATION	llama3.1:70b	Category assignment.
MODEL_SUMMARISATION	qwen2.5-coder:32b	Four-part summary. (Brief specifies <code>command-r-plus</code> ; see 3.10.)
MODEL_SCORING	deepseek-r1:70b	Industrial-relevance score.
MODEL_QA	llama3.1:70b	Summary/score consistency check.
CONTACT_EMAIL	tira25@student.bth.se	Used in polite-pool headers to the paper sources.
CORE_API_KEY	empty	Optional key for the CORE source.
PDF_DIR / TEXT_DIR	...\Software Project\pdfs and ...\pdfs\text	Where PDFs and extracted text are written.
FETCH_DAYS_BACK	7	How far back each source looks (unless the scheduler narrows it).
HTTP_TIMEOUT_SECONDS	30	Per-request network timeout.

SETTING	DEFAULT / CURRENT	MEANING
LOG_LEVEL	INFO	DEBUG / INFO / WARNING / ERROR / CRITICAL .
SITE_BASE_URL / SITE_TITLE	https://dipt.pages.dev	Used in the RSS feed links and title.

5.2 Ollama environment variables

Set once in the Windows user environment during this work (see 3.10). They keep the large models stable on this GPU. If Ollama is ever reinstalled or the profile reset, set them again — without them, scoring runs of more than a handful of papers are likely to fail.

5.3 Agent prompt overrides

Folder `config/agent_prompts/`. An optional file `<agent>.txt` per agent — recognised names: `quality_gate`, `categorisation`, `summarisation`, `scoring`, `qa`. If a file is present its contents replace that agent's built-in prompt on the next run; delete the file to revert. Edit these from the dashboard Settings view rather than by hand, so the placeholder check runs.

6 Current state & demo data

As delivered, the database contains:

ITEM	COUNT	NOTES
Papers approved	28	Fully processed: categorised, summarised, scored, QA-passed. These populate the website data.
Papers at earlier stages	~700	Backlog at parsed / no_pdf / categorised — raw material for future runs.
Categories	47	All active.
Open QA flags	0	Run scripts.qa_demo --escalate to create one for a demo.
Score range on the 28	5.0 – 7.0	Clustered — see 7. Rationales are specific and per-paper.
Website data files	3	papers.json , categories.json , rss.xml — generated, current.

The exact current counts at any time are visible in the dashboard's Pipeline status view.

7 Known limitations & recommended follow-ups

ITEM	STATUS	RECOMMENDED FOLLOW-UP
Scores cluster	The scoring model reads each paper and writes a specific, well-reasoned rationale, but it re-anchors the number rather than spreading it (nearly every paper lands within a two-point band). Rubric tuning and a higher temperature helped only a little.	Redesign the scoring prompt as a <i>comparative</i> judgement ("versus a typical accepted SE paper, is this above or below, and by how much?"), or use a non-reasoning model with a forced distribution. Editable live from the Settings view.
<code>command-r-plus</code>	The brief's summarisation model does not load on the current GPU allocation.	Raise the GPU's memory ceiling (BIOS UMA / VRAM split), then set <code>MODEL_SUMMARISATION=command-r-plus</code> in <code>.env</code> . Until then <code>qwen2.5-coder:32b</code> is a good substitute.
Website build	Code and data complete; never built (no Node.js here).	On a machine with Node: <code>npm ci && npm run build</code> in <code>website/</code> , then deploy <code>website/out/</code> .
Scheduler migration	The <code>fetch_log</code> table needs the database owner's password, which was not available.	Run <code>migrations/001_support_tables.sql</code> as <code>postgres</code> . Only affects the scheduler's run history and incremental fetch.
Dashboard schedule daemon	Does not restart after a machine reboot.	For production, use the Windows Task Scheduler route in <code>docs/SCHEDULING.md</code> instead of the dashboard toggle.
GPU throughput	Scoring is ~2–3 min/paper; a 25-paper full run is ~2–3 hours.	Expected on this hardware. Keep per-run caps modest, or schedule overnight.

8 File-by-file reference

DIPT/ – THE PIPELINE

<code>pipeline.py</code>	Command-line entry point; wires everything together; the <code>all</code> and <code>stream</code> orchestration.
<code>config.py</code>	Validated settings loaded from <code>.env</code> .
<code>exceptions.py</code>	The <code>DIPTError</code> exception family.
<code>logging_config.py</code>	Central logging set-up.
<code>prompt_store.py</code>	File-backed admin prompt overrides.
<code>scheduler.py</code>	Scheduled full-chain runner with fetch-window narrowing and run logging.
<code>site_export.py</code>	Database → <code>website/data/*.json</code> + <code>rss.xml</code> .
<code>models/schemas.py</code>	All typed data models and the status/source enums.
<code>database/connection.py</code>	Threaded connection pool + commit/rollback context manager.
<code>database/repository.py</code>	Every SQL statement in the system.
<code>agents/fetch_agent.py</code>	Fetch stage (with the new save cap).
<code>agents/parser_agent.py</code>	Parse stage — PDF download + text extraction.
<code>agents/categorisation_agent.py</code>	Categorise stage.
<code>agents/summarisation_agent.py</code>	Summarise stage.
<code>agents/scoring_agent.py</code>	Score stage.
<code>agents/qa_agent.py</code>	QA stage — auto-fix then escalate.
<code>agents/quality_gate.py</code>	Fail-closed SE-relevance classifier used by Fetch.
<code>agents/llm_client.py</code>	The one Ollama wrapper — retries, token cap, JSON mode.
<code>sources/*.py</code>	One module per paper source (arXiv, OpenAlex, ...), all sharing one interface.

<code>app.py</code>	The Streamlit app — all six views.
<code>data.py</code>	Cached repository connection; the agent-prompt registry.
<code>runner.py</code>	Background-run manager for the Run pipeline view.
<code>schedule.py</code>	Automated-schedule control — daemon on/off, interval, status.

SCRIPTS/ — WRAPPERS & TOOLS

<code>run_stage.py</code>	Runs one stage and marks its log when done (used by the dashboard).
<code>run_scheduled.py</code>	Runs one full scheduled cycle and marks its log.
<code>scheduler_daemon.py</code>	The loop behind the dashboard Schedule view.
<code>qa_demo.py</code>	Reproduces the two QA modes on demand.
<code>rescore.py</code>	Rewinds approved/scored papers to <code>summarised</code> so scoring can be re-run after prompt tuning.

WEBSITE/ — THE PUBLIC SITE

<code>pages/index.js</code>	Home: search, filters, paper cards.
<code>pages/papers/[id].js</code>	One paper's detail page.
<code>pages/categories/index.js</code> , <code>[slug].js</code>	Category index and per-category pages.
<code>lib/papers.js</code>	Reads the exported JSON at build time.
<code>components/</code> , <code>styles/</code>	Layout, paper card, global CSS.
<code>data/*.json</code> , <code>public/rss.xml</code>	Generated by <code>site_export</code> ; the site's only data source.

SUPPORTING

migrations/001_support_tables.sql	Creates the optional <code>fetch_log</code> table. Run as the DB owner.
config/agent_prompts/	Where saved prompt overrides live (one <code>.txt</code> per agent).
runs/	Created at runtime — per-run and daemon state and logs.
docs/SCHEDULING.md	OS-scheduler setup (Windows Task Scheduler, cron, launchd).
docs/HANDBOOK.html	This document.
README.md	Short project README; points here for detail.
tests/	162 tests; all external calls mocked.

9 Troubleshooting

SYMPTOM	LIKELY CAUSE & FIX
A stage fails immediately with "timed out waiting for llama-server to start"	The model could not load into GPU memory. Restart Ollama (Start menu → Ollama), confirm the five environment variables from 3.10 are set, and re-run. If it persists, run fewer papers per run.
Scoring works for a few papers then every one fails fast with "No usable score"	GPU memory exhaustion after repeated 70B loads. Restart Ollama and re-run; the environment variables above make this far less likely. The un-scored papers are still at summarised — just run score again.
Summaries come back as one phrase repeated many times	A model degrading on a long or awkward prompt. This is why summarisation uses qwen2.5-coder:32b in JSON mode; if you changed the model or prompt, revert it.
Dashboard shows "Could not connect to the database"	PostgreSQL is not running, or .env has the wrong password/host. Test with psql -U postgres -d dipt -c "SELECT 1".
Dashboard starts but a chart or table view errors	Should not happen — the dashboard avoids the blocked charting component. If a fresh dependency install pulled it back in, it is harmless; the views render as Markdown tables.
"Prompt editing needs..." or run-history panel is empty	The optional migration was not run. Prompt editing actually works without it (files, not a table); only the scheduler's history needs migrations/001_support_tables.sql.
Port 8501 already in use	Another Streamlit is running, or a previous one did not exit. Add --server.port 8600, or close the other process.

SYMPTOM	LIKELY CAUSE & FIX
Pipeline status shows nothing / no approved papers	Nothing has been run yet, or everything is still mid-pipeline. Run the stages (4.8) or check the counts per status.
The Schedule view says "ON but the daemon is not running"	The machine rebooted or the daemon was killed. Click "Enable / restart". For a permanent setup, switch to Task Scheduler.
Website build fails / <code>npm</code> not found	Node.js is not installed on this machine. Build on a machine that has it.
Tests fail after a change	<code>python -m pytest tests/ -v</code> shows which. All external calls are mocked, so a failure is in the code, not the environment.

10 Glossary

Agent	A small single-purpose program (a Python class with a <code>run()</code> method) that handles one pipeline stage.
Pipeline stage	One step on the assembly line: Fetch, Parse, Categorise, Summarise, Score, or QA.
Status	The single field on each paper that records how far it has travelled down the line.
Quality gate	A fast LLM yes/no check that decides whether a fetched paper is genuinely SE research. Fails closed — on any doubt or error it answers no.
Ollama	The local service that hosts and runs the language models on <code>localhost:11434</code> . No data leaves the machine.
Model tag	The name Ollama uses for a specific model, e.g. <code>llama3.1:70b</code> — family, then size.
JSON mode	An Ollama option that forces a model's output to be syntactically valid JSON. Used to make the local models' structured output reliable.
Reasoning model	A model (here <code>deepseek-r1:70b</code>) that writes a private "thinking" passage before its answer. That passage is stripped before parsing.
KV cache / context length	Memory a model uses per request, proportional to how much text it holds in view. Shrinking both was the fix for the GPU memory failures.
Quantisation (e.g. <code>q8_0</code>)	Storing model numbers at lower precision to use less memory, with little quality loss.
Streaming mode	Processing one paper all the way through the LLM stages before starting the next, so approved papers appear sooner.
Repository	The single class that holds every SQL statement; agents call its methods instead of writing SQL.
Daemon	A long-running background process. Here, the one the dashboard Schedule view starts to run the pipeline on an interval.
SWEBOK	The Software Engineering Body of Knowledge; its v4 knowledge areas make up 18 of the 47 categories.

DIPT System Handbook — prepared as a complete handover of the work delivered. For the short version see `README.md`; for scheduler setup see `docs/SCHEDULING.md`; for the dashboard specifics see `dashboard/README.md`. Run `python -m pytest tests/ -v` at any time to confirm the system is intact.